

Location Approximation

Overview

10 Feb 2026

- how to extend linear least squares to multivariable and multidimensional fits
- how to use a local least squares fit to create LOESS fits to come up with a non-parametrized least squares approximation
- how to interpolate data in higher dimensions onto a uniform grid for analysis and visualization
- how to compute eigenvalues and eigenvectors and diagonalize matrices using numerical libraries. Discuss the power method to find the largest eigenvalue

LOESS: Locally estimated scatterplot smoothing

Multiple Variables

Multivariable Fit

- so far **fits** have been of the **form**

$$y_i = f_p(x_i)$$

- where **(x, y)** is the **input data** and **p** is a **vector of parameters**.

$$y_i \in \mathbb{R}, \quad x_i \in \mathbb{R}, \quad p \in \mathbb{R}^r$$

- the functional **form** has been a **function of x only**

$$f_p(x) = a + bx$$

$$p = (a, b)$$

$$f_p(x) = a + bx + cx^2$$

$$p = (a, b, c)$$

- **multi-variable** functions are **fitting** functions of the form

$$z = f_p(x, y) = a + bx + cy$$

Formulation

AUTOMOTIVE	CALVES	HOGS	SHEEP	COST
34.327	5.291	8.288	10.649	27.908
12.801	4.658	5.751	14.375	65.434
6.186	6.223	3.878	9.441	41.272
11.983	3.212	0.939	0.062	43.215
5.743	3.080	0.430	0.062	22.165
8.021	4.348	0.189	2.158	38.412
7.889	0.636	0.018	0.009	34.730
3.043	4.095	0.054	0.805	16.576
12.120	3.911	0.314	4.695	27.695
5.094	4.035	0.587	3.047	17.350
1.048	4.356	0.307	0.485	10.484
1.069	3.000	0.808	1.637	27.913
8.470	5.300	3.151	18.247	79.356
14.308	4.791	0.780	5.833	101.334
5.747	4.933	1.304	16.344	47.137
7.937	3.043	1.384	5.324	47.147
7.531	4.900	3.860	3.509	49.945
9.881	4.954	1.000	1.308	45.843
8.097	3.005	1.378	3.138	48.494

ChatGPT 5.2 1

- linear model

$$\text{Cost} = b + a_1 \text{ Cattle} + a_2 \text{ Calves} + a_3 \text{ Hogs} + a_4 \text{ Sheep}$$

- in terms of a vector input with five parameters

$$f(x) = b + a_1 x_1 + a_2 x_2 + a_3 x_3 + a_4 x_4$$

- handled the same way as if you had a polynomial fit

$$f(x) = b + a_1 x^1 + a_2 x^2 + a_3 x^3 + a_4 x^4$$

Julia

```
julia> D
19×5 Matrix{Float64}:
 3.437  5.791  3.268  10.649  27.698
12.801  4.558  5.751  14.375  65.634
 6.136  6.223  3.175   2.811  47.172
11.685  3.212  0.639   0.694  49.295
 5.733  3.222  0.534   2.052  24.115
 3.021  4.348  0.839   2.356  33.612
 1.689  0.634  0.318   2.209   9.512
 2.339  1.895  0.610   0.605  14.755
 1.025  0.834  0.734   2.825  10.57
 2.936  1.419  0.331   0.231  15.394
 5.049  4.195  1.589   1.957  27.843
 1.693  3.602  0.837   1.582  17.717
 1.187  2.679  0.459  18.837  20.253
 9.733  3.951  3.78    0.524  37.465
14.325  4.3    10.781  36.863 101.334
 7.737  9.043  1.394   1.524  47.427
 7.538  4.538  2.565   5.109  45.944
```

```

9.211  4.994  3.081  3.681  45.945
8.697  3.005  1.378  3.138  48.494

```

```
julia> A = [ones(19,1) D[:,1:4]];
```

```
julia> p = D[:,5];
```

```

julia> c = A \ p
5-element Vector{Float64}:
 2.28842457684378
 3.21552480274912
 1.6131476138283969
 0.8148494909933895
 0.8025786215025017

```

Here, **A** is the **design matrix** with a **bias column** and **four predictors**,

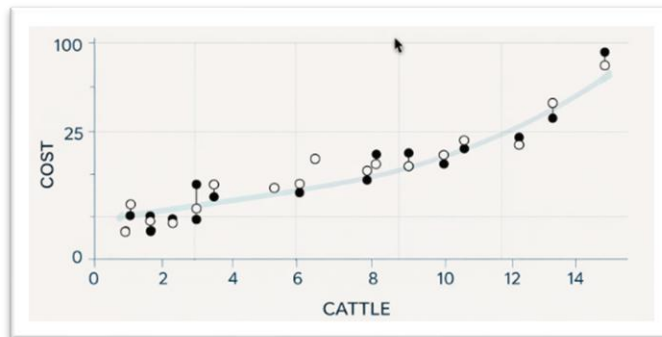
p is the **response vector**, and **c** contains the **least-squares regression coefficients**.

Residual

- **residual** is **computed** the **same** way

$$y \approx Ac \Rightarrow r = y - Ac$$

- input **data** is **four dimensional** but can be **projected** onto **one axis** at a time **for display** purposes
- **dark points** are **data**, **white fit**



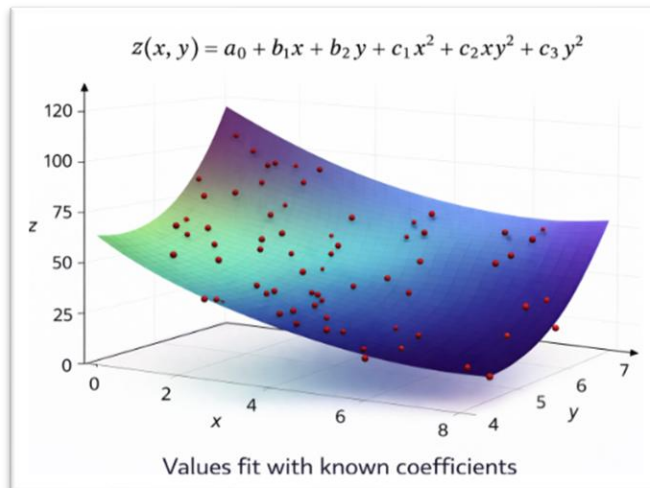
ChatGPT 5.2.2

Spatial

- **values** given in **space**, **fit** with

$$z(x, y) = a_0 + b_1x + b_2y + c_1x^2 + c_2xy + c_3y^2$$

- **test** on data with **known coefficients** and **added noise**



ChatGPT 5.2.3

Julia

```
julia> D
100×3 Matrix{Float64}:
 0.798487  0.692338 -6.78094
-0.388154 -0.325195 -0.200116
 0.780487  0.693793  2.52178
 0.305096 -0.99705  6.87936
 ⋮
-0.869882  0.406619  1.99625
 0.589563  0.109922  2.85972
 0.22768   0.474297  3.69185

julia> X = D[:,1]; y = D[:,2]; val = D[:,3];

julia> A = [ones(size(X)) x y x.^2 y.^2];

julia> A \ val
6-element Vector{Float64}:
-1.0624840106609321
 2.072135382571392
 3.045593968296301
 0.502595092034594
 1.885286192068896
 0.992137812166206
```

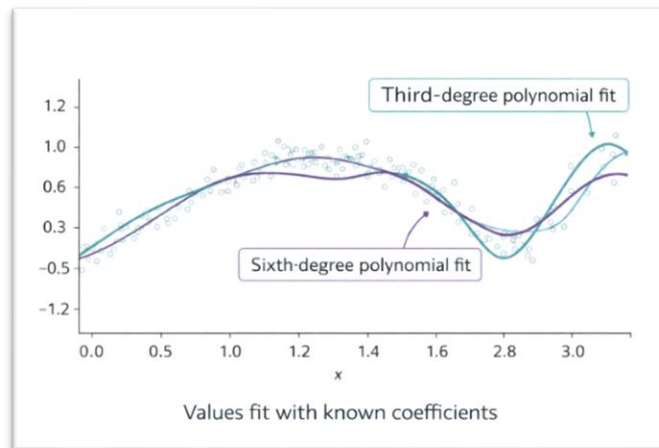
Here, **A** is the **quadratic design matrix** constructed from **x** and **y**, and the **vector** returned by **A \ val** contains the **least-squares estimates**

of the **coefficients** for the **quadratic surface** model.

Local Approximations

Simplifying for Complicated Function

- when dealing with **large functions**, it is **difficult** to find parametrized **fit** that **works** well **globally**
- 3rd** and **6th** degree polynomial fit

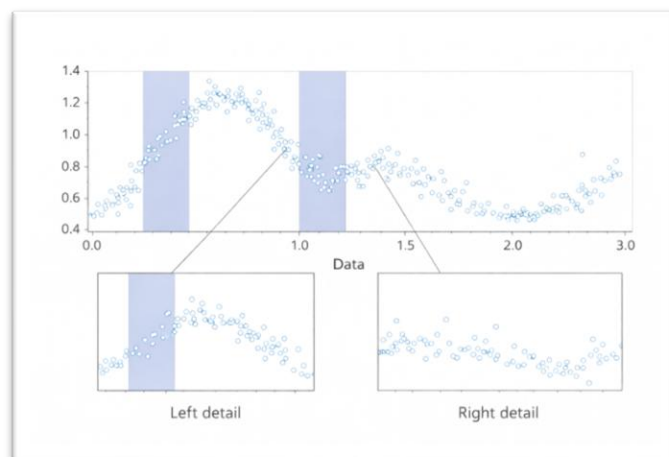


ChatGPT 5.2

- this **pattern** only **gets worse** as polynomial **degrees rise**

Local Regression Scatterplot Smoothing

- separate graph** into chunks that can be **graphed** with **lower degree** polynomials



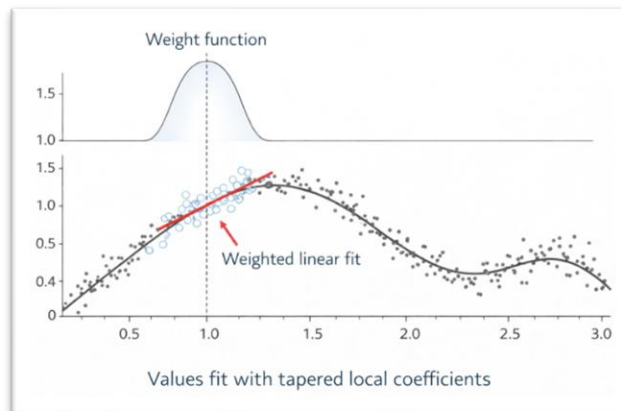
ChatGPT 5.2

- concatenate** smaller-range, lower-degree **graphs**

Details

- for **every x** value, we do a **linear fit** of a **windowed portion** of the data
- apply** a **weight** to that **window**, tapered down to **0**
- do a **least squares** fit of that **sub-set** of the **data**

- **evaluate the fit at the point**



ChatGPT 5.2

Julia

```
julia> D
67×3 Matrix{Float64}:
 0.67  0.80302  8.58312e-5
 0.68  0.75711  0.00211704
  ⋮
 1.32  1.2837   0.00211704
 1.33  1.3288   8.58312e-5

julia> x = D[:,1]; y = D[:,2]; w = D[:,3];

julia> W = Diagonal(sqrt.(w));

julia> A = [ones(size(x)) x];

julia> c = (W*A) \ (W*y)
2-element Vector{Float64}:
 0.2330148228976884
 0.85783827542484

julia> c[1] + c[2]*1
1.0917536504401724
```

This **computes** a **locally weighted** linear **regression** using a **tapered weight** function

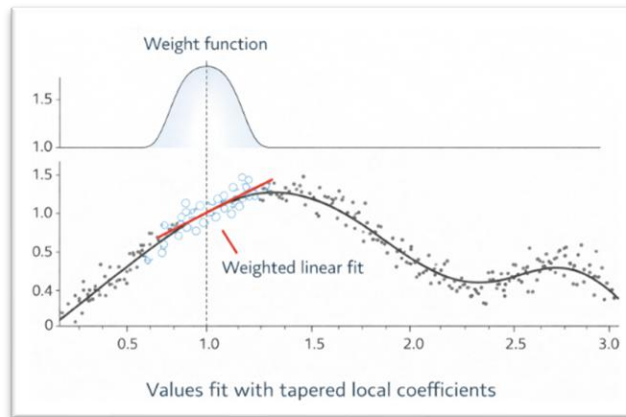
centered at $x = 1$, and evaluates the **fitted model** at that **point**

Weight Function

- **not** a **unique** weight **function**, but a **common** one is the **cubic**

$$\left(1 - \left(\frac{|x - c|}{w}\right)^3\right)^3$$

- w is a **parameter**



ChatGPT 5.2

Julia

```
julia> x = D[:,1]; y = D[:,2];

julia> w = ((1) .- abs.((x .- 1)./0.335).^3).^1.5;

julia> W = Diagonal(w);

julia> A = [ones(size(x)) x];

julia> c = (W*A)\(W*y)
2-element Vector{Float64}:
 0.2330148228976884
 0.8587388275424834

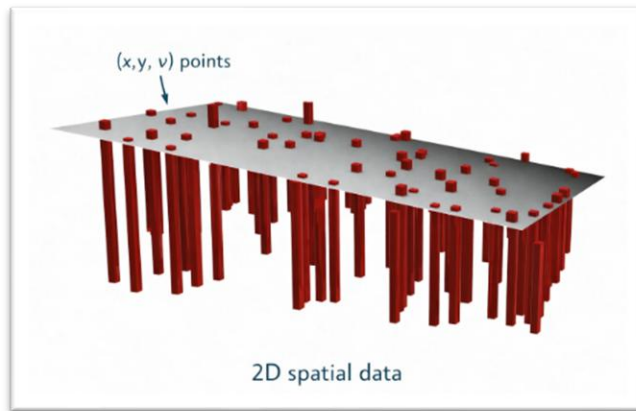
julia> c[1] + c[2]*1
1.091753650440172
```

This computes a **locally weighted linear regression** using a **tapered weight function** **centered** at $x = 1$, and **evaluates** the **fitted model** at that **point**

Multiple Dimensions**Two Dimensions**

- **spatial data** is a special case of **multivariable input data** consisting of a list of (x, y) points with a value v at each point
- first **approximation** is a **linear fit**

$$f(x, y) = a + bx + cy$$
- this is a **standard multivariate fit**



ChatGPT 5.2

Julia

```

julia> D
100×3 Matrix{Float64}:
 0.89652  0.69851  2.22423
 0.16441  0.47566  1.55573
 0.68489  0.23571  0.59624
 0.88108  0.15748  1.17562
 ⋮
 0.44704  0.93056  0.27164
 0.20344  0.47266  1.47501
 0.56538  0.99248  2.11564
 0.34011  0.74739  1.09583

julia> x = D[:,1]; y = D[:,2]; v = D[:,3];

julia> A = [ones(size(x)) x y];

julia> A \ v
3-element Vector{Float64}:
 0.82242157222637813
 0.988772946529357
 1.2541980858432

```

Local Fit

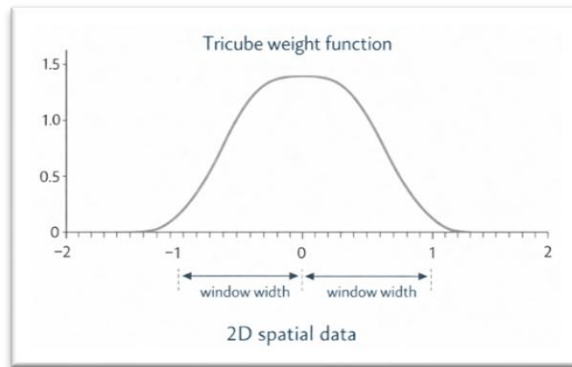
- use a local fit to get a value for a point based on the surrounding points
- the width function

$$\left(1 - \left(\frac{d}{w}\right)^3\right)^3$$

- d is the distance from the center, w is the radius of the region

This is the **tricube weight function** expressed in terms of **distance d** and **window width w**

Typically **used** with the **implicit condition** $|d| < w$ (weight = 0 outside the window)



ChatGPT 5.2

Julia

```
julia> D
146×3 Matrix{Float64}:
-0.386386 -0.880319 -7.8
-0.392838 -0.61673  -1.9
 0.273446  0.582663 -14.8
 ⋮
 0.49234   0.419114 -15.5
 0.463917  0.374538 -13.8

julia> x = D[:,1]; y = D[:,2]; val = D[:,3];

julia> d = norm.(x, y);

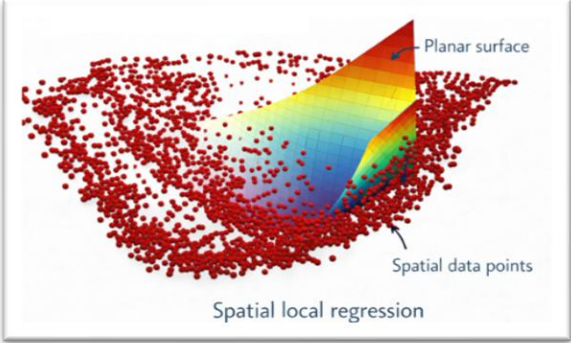
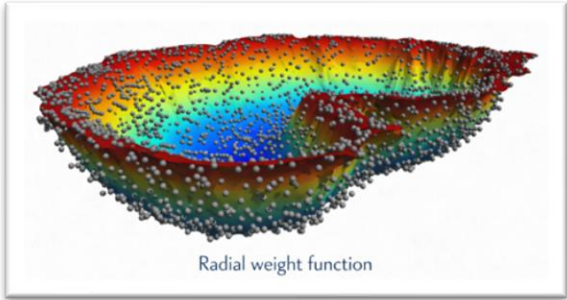
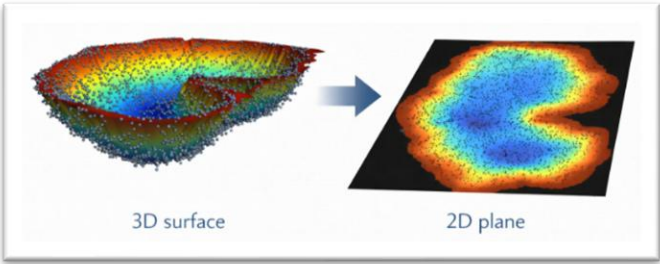
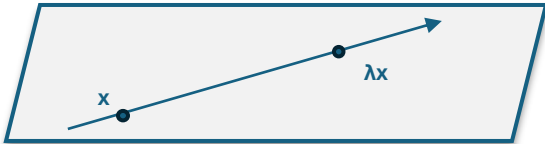
julia> w = (1 .- abs.(d ./ 0.8).^3).^1.5;

julia> W = Diagonal(w);

julia> A = [ones(size(x)) x y];

julia> (W*A) \ (W*val)
3-element Vector{Float64}:
-9.673136939747274
-4.650837446286345
-7.950323134280955
```

Here, **d** is the **radial distance** from the **origin**, **w** is a **tricube weight** function with window **width 0.8**, **A** is the local **planar design matrix**, and the **final vector** gives the weighted **least-squares surface fit** coefficients

Overlay the Function	$f(x, y) = -9.67 - 4.61x - 7.95y$ 
Other Weights	$w(r) = \frac{1}{r^2 + r^4} = \frac{1}{r^2(1 + r^2)}$ 
2D Fit	render 3-D surface on a 2-D plane 
Eigenvalues and Eigenvectors	
Mathematical Statement	- the mathematical statement is pretty straight forward $A x = \lambda x, \quad x \neq 0$ x is an eigenvector of the matrix A λ is an eigenvalue A has to be a square matrix (A and x might be complex numbers)
Geometric Interpretation	geometrically, A scales x by the number λ 

Quick Refresh

- the **standard mathematical approach** to find **eigenvalues** and **eigenvectors**

- **define** the characteristic **equation**

$$p(\lambda) = \det(A - \lambda I)$$

$$p(\lambda) = \det\left(\begin{bmatrix} -4 & -2 \\ -3 & 1 \end{bmatrix}\right) - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \det\left(\begin{bmatrix} -4-\lambda & -2 \\ -3 & 1-\lambda \end{bmatrix}\right) = \lambda^2 + \lambda - 10$$

- If **A** is an $n \times n$ **matrix**, this is an **n-th degree polynomial** in λ .
- The **eigenvalues** are the **roots** of this **polynomial**.

$$\lambda^2 + 3\lambda - 10 = (\lambda + 5)(\lambda - 2) \rightarrow \lambda_1 = -5, \lambda_2 = 2$$

- the **eigenvector** is a **solution** to the **linear system**

$$(A - \lambda I)x = 0$$

$$0 = \begin{bmatrix} -4 - (-5) & -2 \\ -3 & 1 - (-5) \end{bmatrix} x = \begin{bmatrix} 1 & -2 \\ -3 & 6 \end{bmatrix} x \rightarrow x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

- the **second eigenvector** is similar

$$0 = \begin{bmatrix} -4 - (2) & -2 \\ -3 & 1 - (2) \end{bmatrix} x = \begin{bmatrix} -6 & -2 \\ -3 & -1 \end{bmatrix} x \rightarrow x = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$$

- the **eigenvector** is **not unique**; you can **scale** by any **constant**
- all **eigenvectors** that **match** the **dimension** are placed in the **matrix**

$$S = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix}$$

- this **matrix** has the following **property**

$$AS = SA \quad \Lambda = \begin{bmatrix} -5 & 0 \\ 0 & 2 \end{bmatrix}$$

Julia

```
julia> A = [-4 -2; -3 1]
2×2 Matrix{Int64}:
-4  -2
-3   1

julia> A*[2,1] - (-5)*[2,1]
2-element Vector{Int64}:
 0
 0

julia> A*[1,-3] - 2*[1,-3]
2-element Vector{Int64}:
 0
 0
```

These **computations verify** that **[2,1]** and **[1,-3]** are **eigenvectors**

of **A** with **eigenvalues -5** and **2**, respectively, and that **A = SAS⁻¹**

	<pre>0 0</pre>
(Quick?) Refresh Cont.	• most of the time this matrix S is invertible $AS = S\Lambda \Rightarrow A = S\Lambda S^{-1}$ $\Lambda = S^{-1}AS$ • you will often get complex numbers and sometimes the matrix is not diagonalizable ○ you can still find eigenvalues ○ but not an invertible matrix S of eigenvectors. • numerically, that last case is very hard to detect and handle.
Julia	<pre>julia> A 2×2 Matrix{Int64}: -4 -2 -3 1 julia> S \ A / inv(S) 2×2 Matrix{Float64}: -4.0 -2.0 -3.0 1.0 julia> inv(S)*A*S 2×2 Matrix{Float64}: -5.0 4.44089e-16 0.0 2.0 julia> A = [-1 5; -2 5] 2×2 Matrix{Int64}: -1 5 -2 5 julia> S = [5 5-3im; 5+3im 3+1im] 2×2 Matrix{Complex{Int64}}: 5+0im 5-3im 5+3im 3+1im julia> Lambda = diagm([2-im, 2+im]) 2×2 Matrix{Complex{Int64}}: 2-im 0+0im 0+0im 2+im</pre> The first example verifies real diagonalization $A = SAS^{-1}$. The second example illustrates a matrix with complex eigenvalues, requiring complex eigenvectors and a complex diagonal matrix Λ

(Quick!) Refresh Cont.	• previous approach will not give numerical solution • it is very difficult to compute the determinant of • analytical solutions ◦ exists for: $n = 2, 3, 4$ ◦ none exist for: $n > 4$ • high degree polynomials are numerically very tricky $(A - \lambda I)x = 0$ the characteristic polynomial. need iterative methods
Basix Idea for Computing	• pick any x, call it: x^0 • apply A over and over $x_1 = Ax_0, x_2 = Ax_1, x_3 = Ax_2, \dots$ • if you start with an eigenvector you get $x_n = \lambda^n x_0$ ◦ if λ is small this goes to 0 ◦ if λ is larger than 1 this goes to infinity • direction is relevant here, not the magnitude, so we normalize each step. $x_1 = \frac{Ax_0}{\ Ax_0\ }, \quad x_2 = \frac{Ax_1}{\ Ax_1\ }, \quad x_3 = \frac{Ax_2}{\ Ax_2\ }, \dots$
Julia	<pre>julia> A = [19 -6; 45 -14]; julia> x = [1,1]; julia> v = A*x; x = v/norm(v) 2-element Vector{Float64}: 0.3867266762507425 0.922194381828531 julia> x/x[1] 2-element Vector{Float64}: 1.0 2.3846153846153846 julia> v = A*x; x = v/norm(v)</pre> This demonstrates the power method. Repeated normalization of $A*x$ causes x to converge in direction to the dominant eigenvector of A, with the ratio x_2/x_1 approaching the eigenvector slope.
How to Call	• function 'eigen' finds eigenvalues and eigenvectors returns: ◦ list of eigenvalues ◦ eigenvectors $S\Lambda S^{-1} = A$ • computation scales like: $O(n^{2.7})$

Julia	<pre> julia> A = rand(2000,2000); @time B = A*A; 0.074313 seconds (2 allocations: 30.518 MiB) julia> A = rand(4000,4000); @time B = A*A; 0.423728 seconds (2 allocations: 122.070 MiB) julia> A = rand(2000,2000); @time f = eigen(A); 2.222281 seconds (22 allocations: 124.208 MiB) julia> A = rand(4000,4000); @time f = eigen(A); 13.800373 seconds (28 allocations: 492.555 MiB, 0.05% gc time) julia> using LinearAlgebra julia> A = [19 -6; 45 -14]; julia> factor = eigen(A) Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}} values: 2-element Vector{Float64}: 1.0000000000000053 3.9999999999999993 vectors: 2×2 Matrix{Float64}: 0.316228 0.371391 0.948683 0.928477 julia> Lambda = diagm(factor.values) 2×2 Matrix{Float64}: 1.0 0.0 0.0 4.0 julia> S = factor.vectors; julia> S*Lambda*inv(S) - A 2×2 Matrix{Float64}: 7.10543e-15 -3.55271e-15 2.84217e-14 -1.06581e-14 </pre>	shows eigenvalue computation is significantly more expensive than matrix multiplication.
What Can Happen	- For a non-symmetric matrix, eigenvalues and eigenvectors can be complex - For matrices that are not diagonalizable, the return is invalid	The second example verifies diagonalization: $A = SAS^{-1}$, with small numerical error due to floating-point arithmetic

Julia

```
julia> A = [2 1; 0 2]
```

```
2×2 Matrix{Int64}:
```

```
2  1
0  2
```

```
julia> eigen(A)
```

```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
```

```
values:
```

```
2-element Vector{Float64}:
```

```
2.0
2.0
```

```
vectors:
```

```
2×2 Matrix{Float64}:
```

```
1.0 -1.0
0.0  4.44089e-16
```

matrix is **defective**: it has **repeated eigenvalues** ($\lambda = 2$) and does

not have two **independent eigenvectors**

```
julia> A = rand(5,5)
```

```
5×5 Matrix{Float64}:
```

```
0.896989 0.928842 0.194495 0.56745 0.445895
0.833862 0.817736 0.784447 0.956141 0.757149
0.089797 0.585985 0.951922 0.342448 0.593538
0.283555 0.569285 0.15656 0.917266 0.58781
0.865177 0.415104 0.207601 0.759322 0.454219
```

```
julia> eigen(A)
```

```
Eigen{ComplexF64, ComplexF64, Matrix{ComplexF64}, Vector{ComplexF64}}
```

```
values:
```

```
5-element Vector{ComplexF64}:
```

```
-0.0630614639921762 + 0.0im
0.21491356519522642 - 0.18293391221310967im
0.21491356519522642 + 0.18293391221310967im
0.8447011083491994 + 0.0im
2.7393605534814863 + 0.0im
```

```
vectors:
```

```
5×5 Matrix{ComplexF64}:
```

```
0.383351+0.0im 0.192263+0.141744im 0.258763+0.0im 0.515498+0.0im ...
-0.335414+0.0im -0.626591-0.297367im -0.105868+0.0im 0.646038+0.0im
0.032539+0.0im -0.146977+0.140434im -0.875045+0.0im 0.320683+0.0im
0.498739+0.0im 0.558948-0.210011im 0.358323+0.0im 0.389948+0.0im
-0.700528+0.0im -0.735823-0.0im 0.166535+0.0im 0.248999+0.0im
```

illustrates that a **general non-symmetric matrix**

may have **complex eigenvalues** and **complex eigenvectors**

```
julia> eigen(A)
```

```
Eigen{ComplexF64, ComplexF64, Matrix{ComplexF64}, Vector{ComplexF64}}
```

```
values:
```

```
5-element Vector{ComplexF64}:
```

	<pre> -0.0630614639921762 + 0.0im 0.21491356519522642 - 0.18293391221310967im 0.21491356519522642 + 0.18293391221310967im 0.8447011083491994 + 0.0im 2.7393605534814863 + 0.0im vectors: 5×5 Matrix{ComplexF64}: 0.383351+0.0im 0.192263+0.141744im 0.258763+0.0im 0.515498+0.0im ... -0.335414+0.0im -0.626591-0.297367im -0.105868+0.0im 0.646038+0.0im 0.032539+0.0im -0.146977+0.140434im -0.875045+0.0im 0.320683+0.0im 0.498739+0.0im 0.558948-0.210011im 0.358323+0.0im 0.389948+0.0im -0.700528+0.0im -0.735823-0.0im 0.166535+0.0im 0.248999+0.0im </pre>
Symmetric Matrices	• symmetric matrices are ○ always diagonalizable ○ always real ○ orthogonal to each other $S^{-1} = S^T$ • normalize the vectors = <i>matrix</i> 'Q'
Julia	<pre> julia> B = rand(5,5); A = B + B' 5×5 Matrix{Float64}: 1.46841 0.217176 1.14759 1.77643 1.84304 0.217176 1.0952 1.14557 0.673613 1.22555 1.14759 1.14557 1.1925 0.83276 1.10775 1.77643 0.673613 0.83276 0.31743 1.02043 1.84304 1.22555 1.10775 1.02043 0.311548 julia> factors = eigen(A) Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}} values: 5-element Vector{Float64}: -1.479744357061945 -0.7384718547725923 -0.018271356454782802 1.2357763366272927 5.385759909664219 vectors: 5×5 Matrix{Float64}: -0.59091 -0.200388 0.0384984 0.552359 -0.551442 -0.358851 -0.676396 0.462094 -0.730869 -0.343058 0.037868 0.0585515 -0.089259 -0.326998 -0.447297 0.339562 0.791801 0.198595 0.231927 -0.405452 0.636644 -0.568774 0.239608 -0.0035335 -0.462338 </pre>

Because A is symmetric,
the eigenvalues are real
and the eigenvectors

form an orthonormal basis,
so $Q^T Q = I$ up to floating-
point error.


```
julia> Q = factors.vectors;
```

```
julia> Q' * Q
```

```
5×5 Matrix{Float64}:
```

```
1.0    1.71193e-16 -3.53161e-17 -3.18443e-17 ...  
1.71193e-16 1.0    -2.86422e-17 2.47198e-17  
-3.53161e-17 -2.86422e-17 1.0    4.60241e-17  
-3.18443e-17 2.47198e-17 4.60241e-17 1.0  
...
```